

Cloud-Based Data Analytics Solution

GRANDVIEW HOTELS — MID-SIZE HOTEL CHAIN

Prepared for: Cloud Computing & Data Analytics Course

Scope: 1,000 users (reception, managers, analysts, guests) across 8 properties

Architecture diagram: `architecture.drawio` (open in draw.io / [Lucidchart](https://lucidchart.com))

Task 1 — Organization's Needs

1.1 Selected Organization

Grandview Hotels is a mid-size hotel chain operating **8 properties** (city hotels + resorts) with a combined ~1,200 rooms. It runs the `booking-system` application (Laravel + MySQL) described throughout this project for reservations, payments, rooms, and staff operations. It employs ~900 staff and serves ~100k guests/year, giving an active system user base of **1,000 users** (internal staff, managers, and data analysts; guests use the public portal).

1.2 What Data Does the Organization Collect?

Data Domain	Examples	Source
Booking & Reservation	Booking number, check-in/out, guests, room assignment, status	Reception, Web Portal
Guest / CRM	Name, email, phone, preferences, loyalty, reviews	Registration, Front Desk
Financial	Payments, invoices, refunds, discounts, revenue	POS, Payment Gateway (Stripe)
Operations	Room status, housekeeping, maintenance, inventory	IoT, Staff App
Marketing	Campaigns, newsletter subs, channel (OTA vs direct)	Website, CRM
External / Market	Local events, weather, competitor rates, seasonality	Third-party APIs

1.3 Who Uses the System?

- **Receptionists** — check-ins/out, walk-in bookings, room status.
- **Hotel Managers** — occupancy, revenue, staffing, discounts.
- **Revenue / Data Analysts** — BI dashboards, forecasting, reporting.
- **Administrators / IT** — user roles, hotels, system config.
- **Guests (external)** — search, book, pay, review via web/mobile portal.

1.4 Current Problems

- **Siloed data:** each property runs reports locally; no group-wide view of occupancy or revenue.
- **Manual reporting:** managers export CSVs to Excel — slow, error-prone, not real-time.
- **No predictive insight:** pricing is static; demand spikes and low-occupancy periods are missed.
- **Scalability ceiling:** on-prem MySQL struggles during peak booking seasons and OTA bursts.
- **Disaster recovery gaps:** nightly backups only; a site failure means hours of downtime and lost bookings.
- **Compliance risk:** guest PII and payment data stored without centralized encryption/audit.

1.5 Why Move to the Cloud?

- **Elastic scale** — absorb seasonal booking spikes without over-provisioning hardware.
- **Centralized analytics** — one warehouse across all 8 properties enables real-time BI.
- **Lower TCO** — no capital spend on servers; pay-per-use pricing.
- **Resilience** — multi-AZ, cross-region backups, 99.95% SLA.
- **Managed security & compliance** — PCI-DSS, GDPR tooling offloaded to the provider.
- **Faster innovation** — serverless + ML services enable dynamic pricing in weeks, not quarters.

Task 2 — Cloud Solution Architecture

The full diagram is in `architecture.drawio`. The solution flows left-to-right:

Client Devices → Internet/CDN → API Gateway → Backend Services → Database/Storage → ETL → Data Warehouse → BI Dashboards.

Layer	Components	Purpose
Client Devices	Reception desktops, manager laptops, staff mobile PWA, self-service kiosks, IoT sensors (energy/access), guest devices, POS terminals	Data capture & consumption at the edge
Internet / CDN	CloudFront CDN, Route 53 DNS, WAF, Application Load Balancer (ALB)	Secure global delivery, DDoS protection, traffic routing
Backend (PaaS)	API Gateway + Auth (JWT/OAuth2), App Services (Booking, Payments, Reports), Serverless Functions (ETL/notifications), Message Queue, ML Service	Business logic, integration, event processing
Database (PaaS)	PostgreSQL/MySQL (relational), Redis (cache), Redshift/BigQuery (warehouse)	Transactional + analytical storage
Storage (PaaS)	Object Storage (S3) for docs/images, Data Lake (Parquet/Iceberg), Backup Vault (cross-region)	Durable, cheap, scalable file & raw-data storage
Analytics & Frontend	ETL Pipelines, BI Dashboards (Power BI/Looker), Admin/Staff Web App, Guest Portal, Automated Reports, Predictive Analytics	Insight delivery to every user role
Private / Hybrid	PII Vault (guest IDs, payment tokens), VPN/Direct Connect, Compliance/Audit node	Keep regulated data on-prem, encrypted tunnel to cloud

Data Flow Example: A guest books on the portal → API Gateway authenticates → Booking Service writes to PostgreSQL and emits an event to the Queue → Serverless Function sends confirmation + writes to Data Lake → Nightly ETL loads into Warehouse → BI shows live occupancy & revenue.

Task 3 — Cloud Service & Deployment Models

3.1 Service Models Used

Model	Used For	Justification
SaaS Software as a Service	Power BI / Looker (BI), Stripe (payments), email/SMS, GSuite/M365	Zero-maintenance analytics & business tools; fastest time-to-value; no servers to patch.
PaaS Platform as a Service	Managed DB (PostgreSQL), App Services / Containers, Object Storage, Serverless, Data Warehouse	Developers focus on booking/revenue logic, not OS/patching; auto-scaling handles peaks.
IaaS Infrastructure as a Service	Virtual network, load balancer, VPN/Direct Connect to on-prem PII vault	Needed only for the hybrid network boundary and compliance tunnel; minimal footprint.

Primary choice: PaaS + SaaS. The hotel is a hospitality business, not a tech company — it should not manage VMs/OS. IaaS is limited to the network edge of the hybrid model.

3.2 Deployment Model

Hybrid Cloud (Public cloud for analytics/APP/BI + Private on-prem vault for regulated PII/payment tokens).

- **Public** hosts the bulk of compute, storage, warehouse, and BI — elastic and cheap.
- **Private/on-prem** holds guest national IDs and payment tokens behind a VPN/Direct Connect tunnel, satisfying PCI-DSS & GDPR data-residency rules.
- **Why hybrid, not pure public:** regulatory and guest-trust requirements mandate keeping sensitive identifiers under direct organizational control. Pure private would forfeit elasticity and BI tooling speed.

Task 4 — Cost Estimation (1,000 Users)

Indicative pricing on AWS-equivalent services, USD/month, 8 properties, ~1,000 active users, ~100k bookings/year.

Item	Specification	Monthly Cost
Hosting (App Services / Containers)	3 env × 2 vCPU / 4GB, auto-scale to ~6 instances at peak	\$420
Relational Database (Managed)	db.t3.medium HA (primary+standby), 50 GB	\$210
Cache (Redis)	1 GB managed node	\$35
Object Storage (S3)	200 GB docs/images + 500 GB backups	\$28
Data Lake + Warehouse	200 GB warehouse, ~1 TB scan/month	\$180
ETL / Serverless + Queue	~5M invocations + pipeline runs	\$60
API / Gateway + CDN	~10M API calls, 2 TB egress/CDN	\$140
BI (SaaS — Power BI Pro)	25 analyst/manager licenses	\$125
Networking (ALB, DNS, WAF, VPN)	Load balancer + WAF + Direct Connect port share	\$95
Monitoring + Backup vault	Logs, metrics, cross-region backup	\$55
Total Estimated		\$1,348 / month

4.1 Why This Design Is Cost-Effective

- **Pay-per-use:** serverless + auto-scaling means the hotel pays for peak only during high season, idle capacity shrinks at night.
- **No CapEx:** eliminates ~\$40k–\$60k upfront server/storage purchase and ongoing DC power/cooling.
- **Managed services:** offloading DB patching, backups, and BI to PaaS/SaaS removes 2–3 FTE ops salaries.
- **Storage tiering:** cold backups and raw lake data use cheap object storage, not expensive DB blocks.
- **Hybrid only where required:** keeping a tiny on-prem PII vault avoids the much higher cost of a full private cloud.
- **ROI:** dynamic pricing + reduced vacancy (even a 2% occupancy lift) recovers the entire monthly bill many times over.

Task 5 — Risk Analysis

Risk 1 — Data Breach / Unauthorized Access

Impact: Guest PII and payment data leak → GDPR/PCI fines, loss of trust, legal exposure.

Mitigation: Encrypt data at rest (KMS) and in transit (TLS); WAF + API Gateway throttling; least-privilege IAM roles; centralized audit logs; store payment tokens/IDs only in the private vault; annual penetration testing and PCI-DSS compliance scans.

Risk 2 — Downtime / Service Outage

Impact: Booking system offline during peak = lost revenue and stranded guests.

Mitigation: Multi-AZ deployment with ALB health checks; managed DB with standby + automated cross-region backups (RPO < 5 min, RTO < 30 min); CDN serves cached static content if origin fails; defined runbooks and 99.95% provider SLA with credits.

Risk 3 — Vendor Lock-in

Impact: High switching cost if provider raises prices or exits a region.

Mitigation: Use containers (portable) and infrastructure-as-code (Terraform); keep data in open formats (Parquet/Iceberg) in the Data Lake; abstract BI via standard SQL so it can move between Redshift/BigQuery/Snowflake; hybrid vault ensures critical data is never 100% provider-bound.

Risk 4 — Internet Dependency

Impact: Property loses connectivity → front desk cannot check guests in/out.

Mitigation: Local kiosk/reception cache with offline mode for check-in/out; redundant ISP links + 4G/5G failover at each property; queue-based sync reconciles local changes once connectivity returns.

Conclusion

The proposed **hybrid cloud data-analytics platform** transforms Grandview Hotels from siloed, manual reporting to a real-time, predictive, group-wide operation. By combining **PaaS + SaaS** on a **public**

cloud with a small **private** compliance vault, the hotel gains弹性 scale, lower TCO (~\$1.35k/month), and the analytics foundation for dynamic pricing and elevated guest experience — while keeping regulated data secure and portable.

Deliverables: this PDF report + `architecture.drawio` (editable in draw.io / Lucidchart).